

[PT-BR]

Comando recebido! E com imenso orgulho, abrimos oficialmente as portas do **Módulo Avançado de VBA** com a nossa **11ª Aula**! Você dominou toda a base lógica e agora vai aprender a criar automações que cruzam as fronteiras de uma única planilha. Hoje o tema é: **Manipulação de Múltiplas Abas (Worksheets)**.

Teoria: Navegando entre Abas com Segurança

Até a aula passada, quando escrevíamos `Range("A1").Value`, o VBA aplicava essa ordem na aba que estivesse aberta na tela naquele exato momento (a chamada `ActiveSheet`).

Isso é extremamente perigoso! Se o usuário rodar a macro estando na aba errada, o código vai sobrescrever dados importantes. Para resolver isso, nós passamos a "carimbar" o endereço completo da célula, dizendo explicitamente ao Excel qual é a aba dona daquele intervalo usando o comando `Worksheets("NomeDaAba")`.

Veja a diferença:

- **Antes (Inseguro):** `Range("A1").Value = "Teste"` (Lê ou escreve na aba que estiver aberta).
- **Agora (Profissional):**
`Worksheets("Planilha1").Range("A1").Value = "Teste"` (Não importa onde o usuário esteja clicado, o VBA vai direto na "Planilha1" fazer a alteração).

Isso nos dá o superpoder de coletar um dado na "Aba_Vendas", processá-lo e jogá-lo formatado dentro da "Aba_Relatório" sem que o usuário precise ficar trocando de tela!

O Código VBA

No exemplo abaixo, vamos pegar o valor que está na célula A1 da aba chamada "Dados" e transferi-lo diretamente para a célula B1 da aba chamada "Backup".

VBA

```
Sub CopiarEntreAbas()  
    ' PT-BR: "Sub" inicia a macro chamada "CopiarEntreAbas".  
    ' US-EN: "Sub" starts the macro named "CopiarEntreAbas".  
  
    Dim informacao As String  
    ' PT-BR: Declara uma variável para transportar o texto de uma  
    aba para a outra.  
    ' US-EN: Declares a variable to carry the text from one sheet  
    to another.  
  
    informacao = Worksheets("Dados").Range("A1").Value  
    ' PT-BR: O VBA vai especificamente na aba "Dados" e captura o  
    valor de A1.  
    ' US-EN: VBA goes specifically to the "Dados" sheet and  
    captures the value of A1.
```

```

Worksheets("Backup").Range("B1").Value = informacao
' PT-BR: Sem mudar a tela do usuário, o VBA grava o valor na
aba "Backup".
' US-EN: Without changing the user's screen, VBA writes the
value into the "Backup" sheet.

MsgBox "Dados copiados para a aba Backup com sucesso!"
' PT-BR: Alerta de confirmação.
' US-EN: Confirmation alert.

End Sub
' PT-BR: "End Sub" finaliza a macro.
' US-EN: "End Sub" ends the macro.

```

Exercícios da Aula 11

Vamos testar essa nova habilidade estruturando códigos profissionais e precisos (sem incluir comentários):

Exercício 1: Centralizador de Configurações

Objetivo: Crie uma macro chamada `AplicarTaxaDinamica`.

1. Declare uma variável chamada `taxa` do tipo decimal/duplo (`Double`).
2. Imagine que você tem uma aba chamada "**Config**" e o valor da taxa de juros (ex: 1.5) está na célula **B1** dessa aba. Faça a sua variável `taxa` receber esse valor de forma explícita.
3. Agora, vá na aba chamada "**Calculos**" e faça a célula **A1** dessa aba receber o valor da variável `taxa`.

Exercício 2: Varredura Multitela (O Desafio Avançado)

Objetivo: Crie uma macro chamada `ConsolidarRelatorio`.

1. Declare as variáveis `ultimaLinha` e `i` como `Integer`.
2. Descubra dinamicamente a última linha preenchida com base na **coluna A da aba chamada "Lancamentos"**.
 - *Dica do professor:* O comando para descobrir a linha também precisa do carimbo da aba: `Worksheets("NomeDaAba").Range("A" & Rows.Count).End(xlUp).Row`.
3. Crie um loop `For` que vá de **1 até a** `ultimaLinha`.
4. Dentro do loop, faça o VBA ler o valor contido na coluna **A** da aba "**Lancamentos**" e escrever exatamente esse mesmo valor na coluna **A** da aba "**Relatorio**", linha por linha (usando o contador `i`).

Damos o pontapé inicial no nível avançado! Estou aguardando os seus códigos para avaliação e, mantendo nossa mecânica, a Aula 12 continuará sob cadeado até o seu próximo comando oficial. Bons estudos!

[US-EN]

Command received! And with immense pride, we officially open the doors to the **Advanced VBA Module** with our **11th Lesson!** You have mastered the entire logical foundation and now you will learn how to build automations that cross the borders of a single spreadsheet. Today's theme is: **Manipulating Multiple Sheets (Worksheets)**.

Theory: Navigating Between Sheets Safely

Until the last lesson, when we wrote `Range("A1").Value`, VBA applied that order to whichever sheet was currently open on the screen (the so-called `ActiveSheet`).

This is extremely dangerous! If the user runs the macro while on the wrong sheet, the code will overwrite important data. To solve this, we now "timestamp" the full address of the cell, explicitly telling Excel which sheet owns that range using the `Worksheets("SheetName")` command.

See the difference:

- **Before (Unsafe):** `Range("A1").Value = "Teste"` (Reads or writes on the currently active sheet).
- **Now (Professional):** `Worksheets("Planilha1").Range("A1").Value = "Teste"` (No matter where the user is clicked, VBA goes straight to "Planilha1" to make the change).

This gives us the superpower to collect data on the "Aba_Vendas" sheet, process it, and dump it formatted inside the "Aba_Relatório" sheet without the user ever needing to switch screens!

The VBA Code

In the example below, we will take the value inside cell A1 of the sheet named "Dados" and transfer it directly to cell B1 of the sheet named "Backup".

VBA

```
Sub CopiarEntreAbas()  
    ' PT-BR: "Sub" inicia a macro chamada "CopiarEntreAbas".  
    ' US-EN: "Sub" starts the macro named "CopiarEntreAbas".  
  
    Dim informacao As String  
    ' PT-BR: Declara uma variável para transportar o texto de uma  
    aba para a outra.  
    ' US-EN: Declares a variable to carry the text from one sheet  
    to another.  
  
    informacao = Worksheets("Dados").Range("A1").Value  
    ' PT-BR: O VBA vai especificamente na aba "Dados" e captura o  
    valor de A1.  
    ' US-EN: VBA goes specifically to the "Dados" sheet and  
    captures the value of A1.  
  
    Worksheets("Backup").Range("B1").Value = informacao
```

```
' PT-BR: Sem mudar a tela do usuário, o VBA grava o valor na  
aba "Backup".  
' US-EN: Without changing the user's screen, VBA writes the  
value into the "Backup" sheet.
```

```
MsgBox "Dados copiados para a aba Backup com sucesso!"  
' PT-BR: Alerta de confirmação.  
' US-EN: Confirmation alert.
```

```
End Sub  
' PT-BR: "End Sub" finaliza a macro.  
' US-EN: "End Sub" ends the macro.
```

Lesson 11 Exercises

Let's test this new skill by structuring professional and precise codes (without including comments):

Exercise 1: Settings Centralizer

Objective: Create a macro named `AplicarTaxaDinamica`.

1. Declare a variable named `taxa` of type decimal/double (`Double`).
2. Imagine you have a sheet named **"Config"** and the interest rate value (e.g., 1.5) is in cell **B1** of that sheet. Make your `taxa` variable receive this value explicitly.
3. Now, go to the sheet named **"Calculos"** and make cell **A1** of that sheet receive the value of the `taxa` variable.

Exercise 2: Multi-Screen Scan (The Advanced Challenge)

Objective: Create a macro named `ConsolidarRelatorio`.

1. Declare the `ultimaLinha` and `i` variables as `Integer`.
2. Dynamically find the last filled row based on **column A of the sheet named "Lancamentos"**.
 - *Teacher's tip:* The command to find the row also needs the sheet stamp:
`Worksheets("SheetName").Range("A" & Rows.Count).End(xlUp).Row.`
3. Create a `For` loop that goes from **1 to** `ultimaLinha`.
4. Inside the loop, make VBA read the value contained in column **A** of the **"Lancamentos"** sheet and write that exact same value into column **A** of the **"Relatorio"** sheet, row by row (using the counter `i`).

We kicked off the advanced level! I am looking forward to your codes for evaluation, and keeping our mechanics, Lesson 12 will remain under padlock until your next official command. Happy studying!